

---

**コンピュータビジョン特論A/B**  
**現代脳計算論**

**Computer Vision A/B**  
**Modern Neural Computation**

---

Keisuke Yoneda, Kanazawa University  
Email: [k.yoneda@staff.kanazawa-u.ac.jp](mailto:k.yoneda@staff.kanazawa-u.ac.jp)

# Schedule (Computer Vision A)

---

2

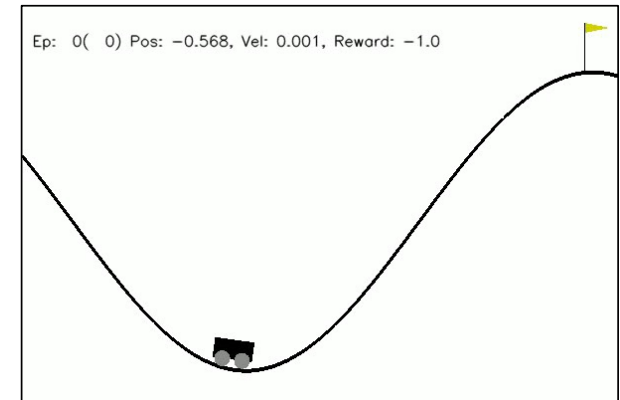
1. 10/3, Introduction (on-demand)
2. 10/10, Local Feature, Corner Detector
3. 10/17, Local Feature, Template Matching
4. 10/24, Supervised Learning (online)
5. 11/7, Convolutional Neural Network
6. 11/14, 3D Reconstruction, Stereo Matching, NeRF
7. 11/21, Recurrent Neural Network, Transformer (on-demand)
8. 11/28, Report

# ○ Schedule (Computer Vision B)

---

3

1. 12/12 Introduction, Evolutionary Neural Network
2. 12/19 Meta Heuristics using OpenGym
3. 12/26 Graph Neural Network
4. 1/9 Reinforcement Learning
5. 1/13 Unsupervised Learning/PCA, Auto-Encoder, VAE
6. 1/23 GAN, Diffusion Model, Image generation, Anomaly Detection
7. 1/30 3D Reconstruction, 3D Gaussian Splatting
8. 2/6 Report



# Computer Vision?

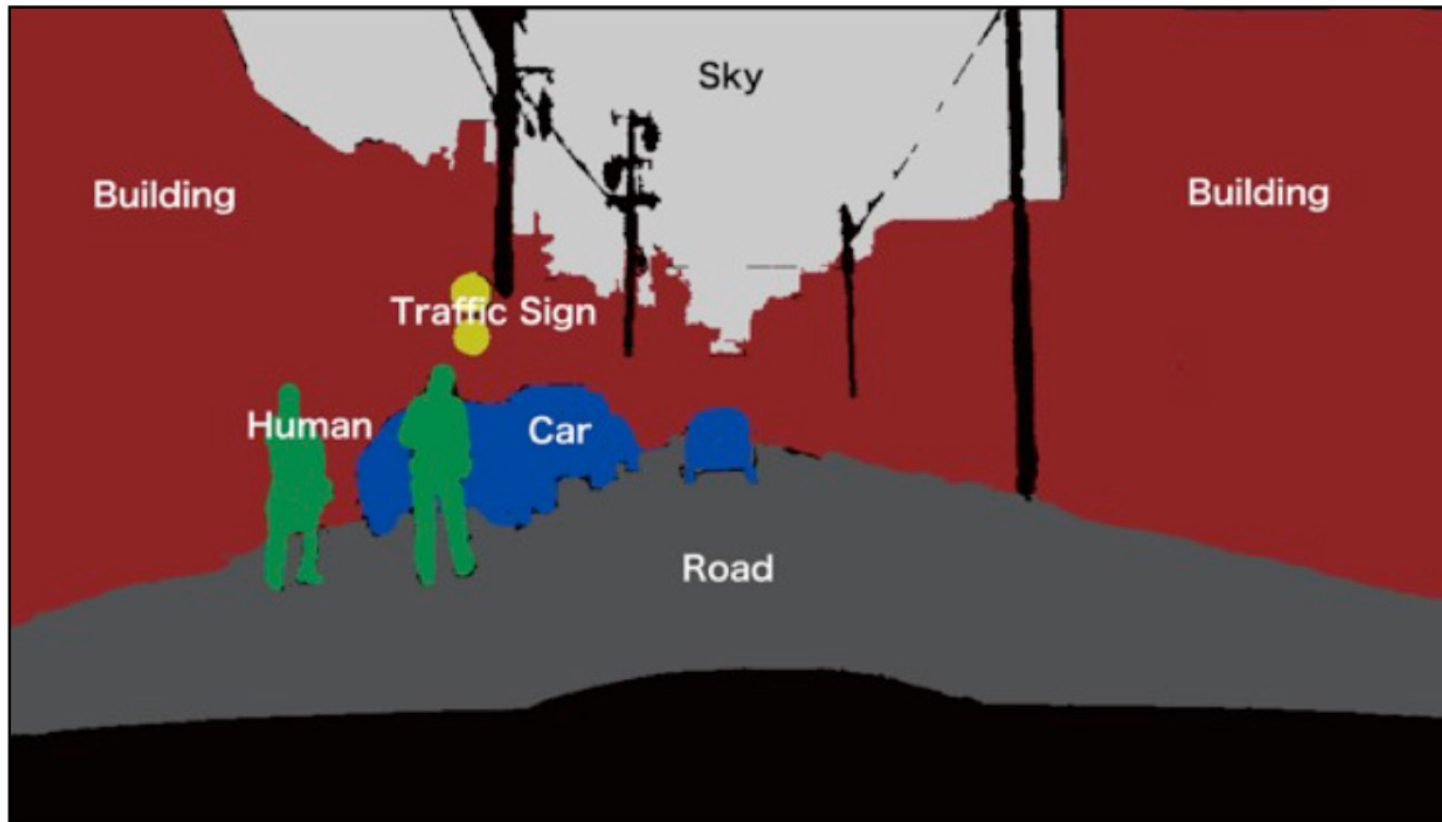
5



[藤吉, 第76回ロボット工学セミナー, 2013]

# Computer Vision?

6

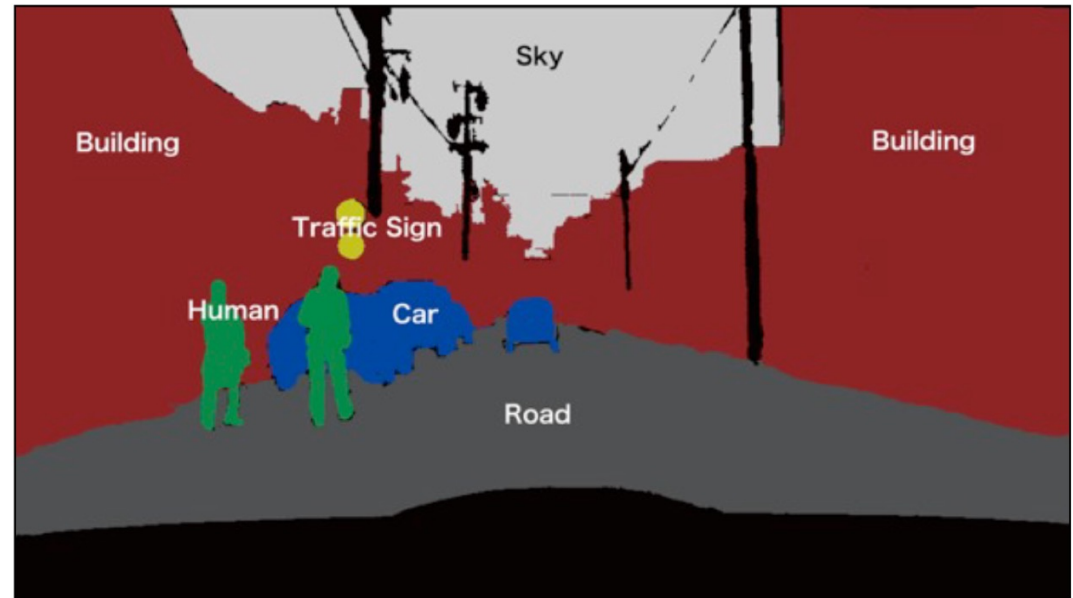


歩行者検出・車両検出・標識認識・白線検出・シーン分割・シーン理解

[藤吉, 第76回ロボット工学セミナー, 2013]

# Computer Vision?

7

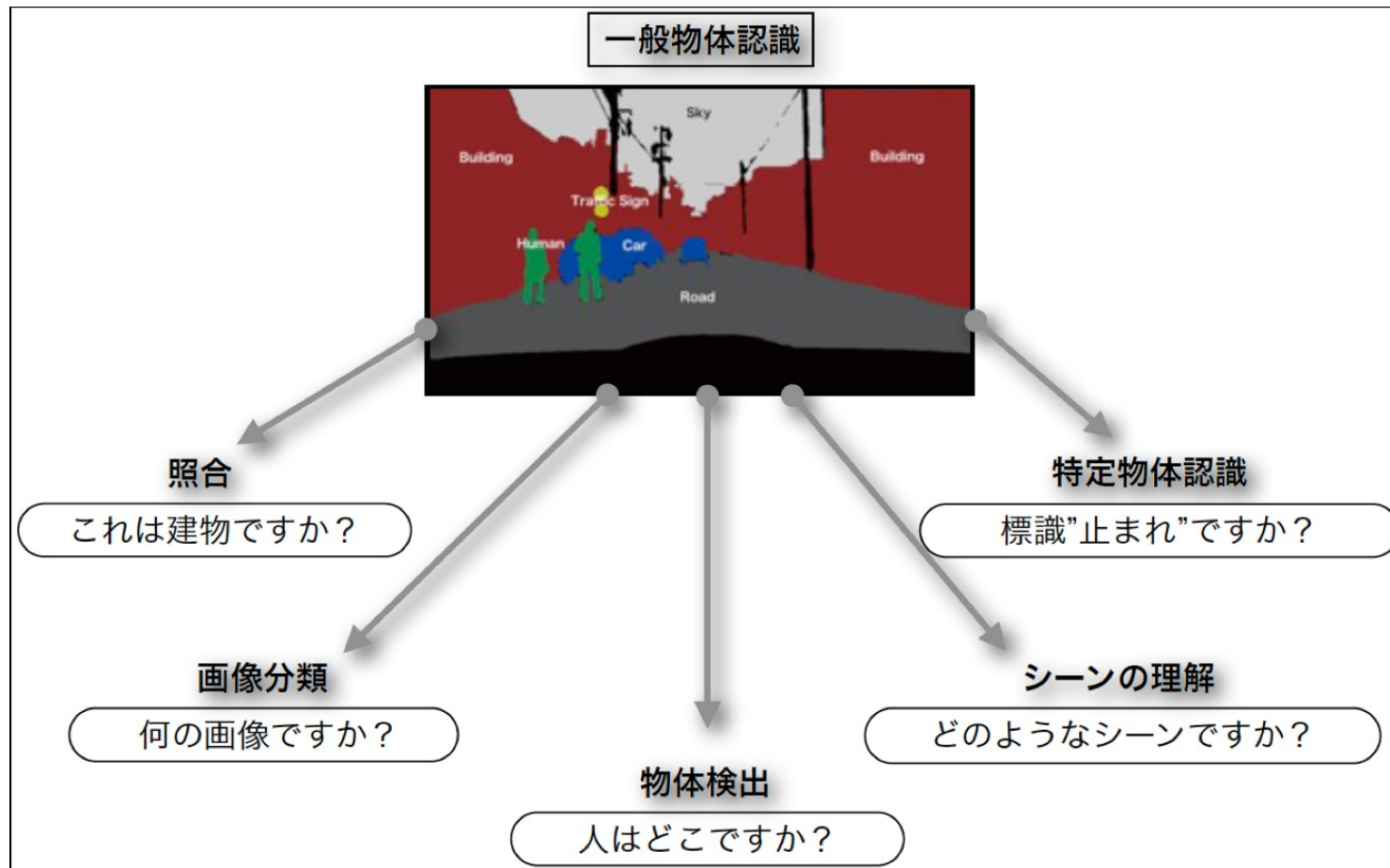


歩行者検出・車両検出・標識認識・白線検出・シーン分割・シーン理解

[藤吉, 第76回ロボット工学セミナー, 2013]

# Computer Vision?

8



[藤吉, 第76回ロボット工学セミナー, 2013]



# Computer Vision/Robot Vision

9

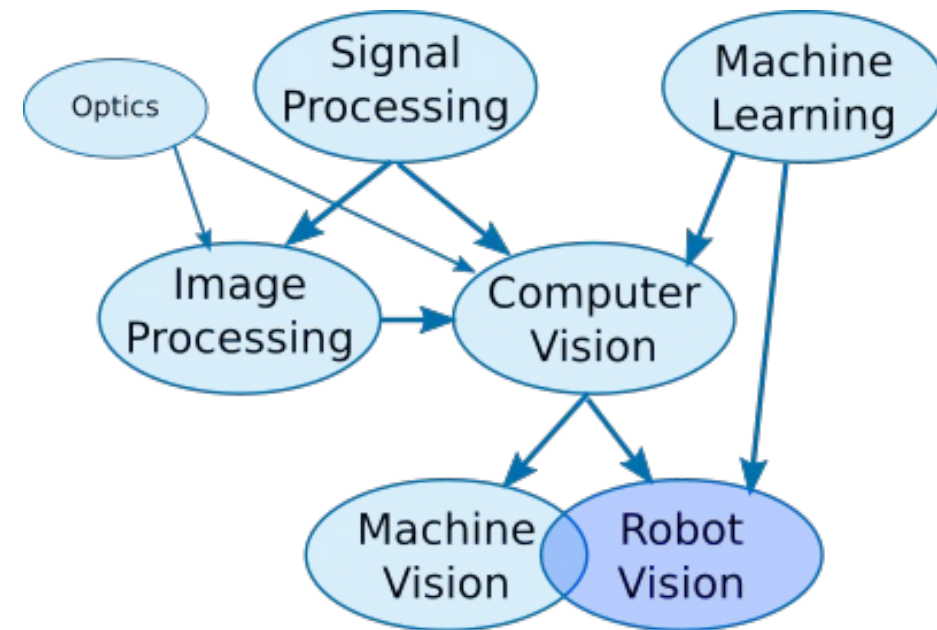
## Computer Vision

- Input: Images (or Sensing Data)
- Output: Information/Features

## Robot Vision

- Input: Images (or Sensing Data)
- Output: Physical Action

| Technique                            | Input                | Output               |
|--------------------------------------|----------------------|----------------------|
| Signal Processing                    | Electrical signals   | Electrical signals   |
| Image Processing                     | Images               | Images               |
| Computer Vision                      | Images               | Information/features |
| Pattern Recognition/Machine Learning | Information/features | Information          |
| Machine Vision                       | Images               | Information          |
| Robot Vision                         | Images               | Physical Action      |



<https://blog.robotiq.com/robot-vision-vs-computer-vision-whats-the-difference>



# Object Recognition

10



Input

Feature Extraction

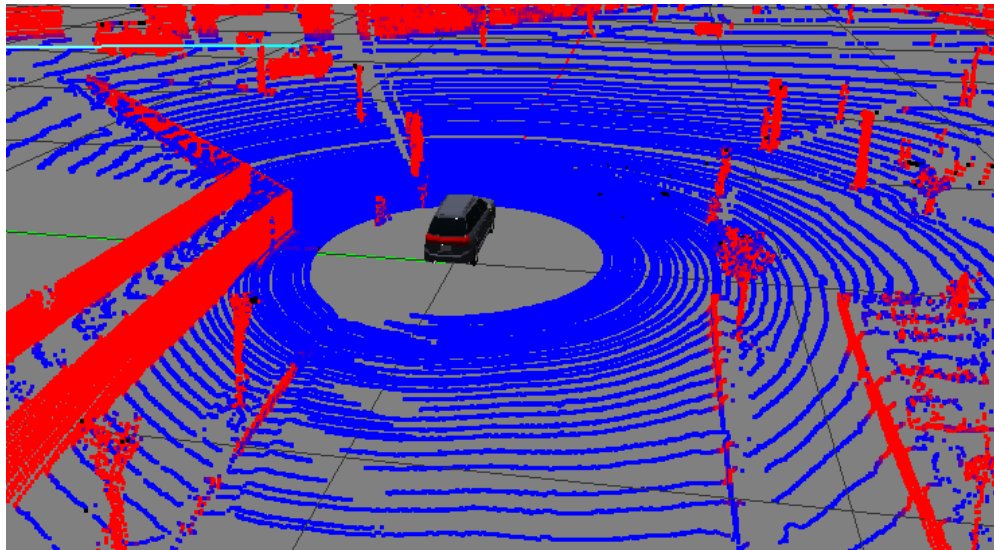
Time-series Processing

Detection/Recognition

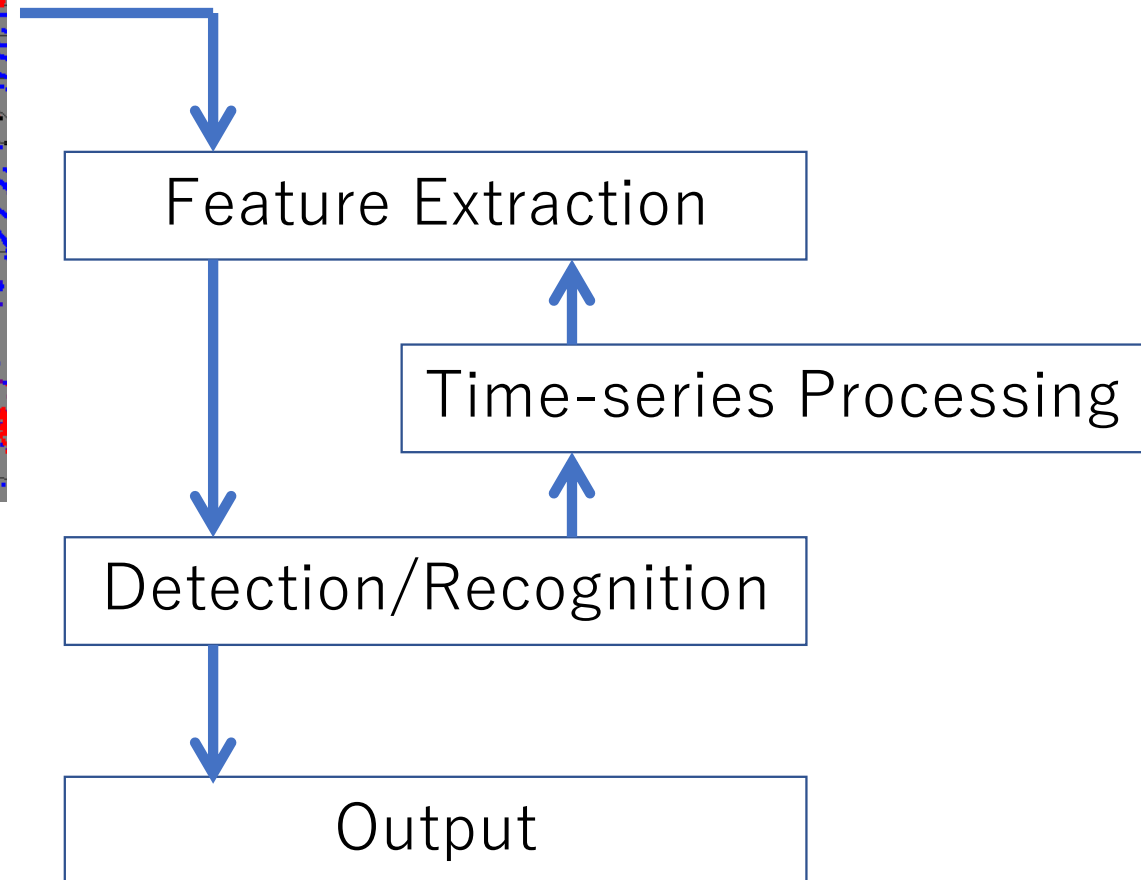
Output

# Object Recognition

11

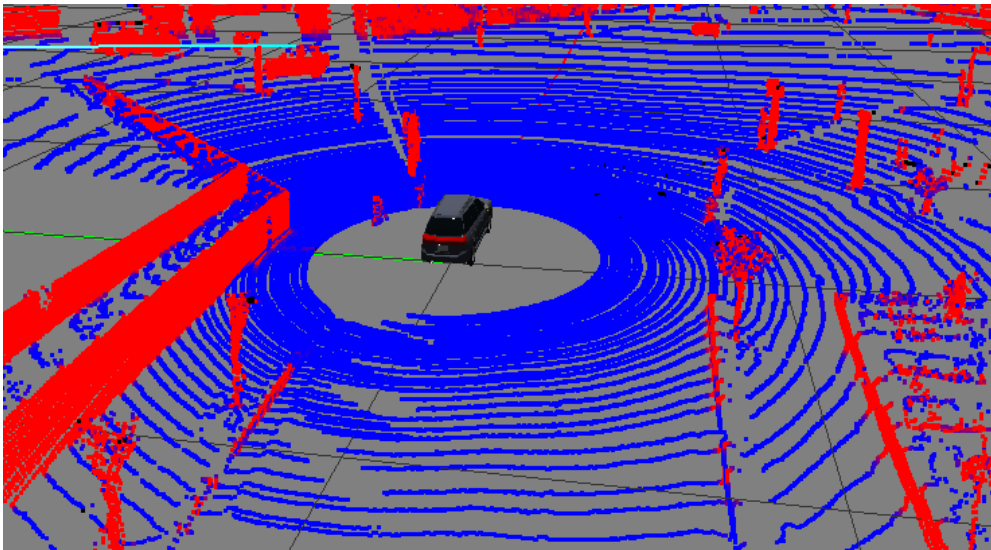


Input

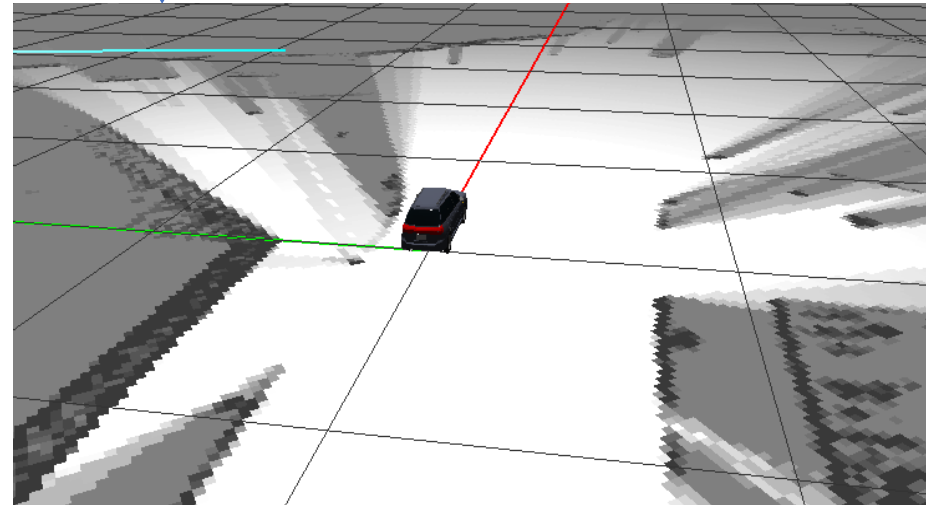
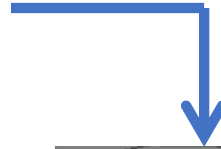


# Object Recognition

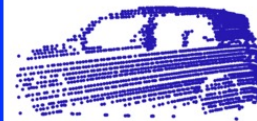
12



Input



自動車



二輪車



歩行者



# Report

---

13

- ❑ 1. Brief summary of the lectures
  - A4, 2~4 pages
- ❑ 2. Implement a program related to computer vision and machine learning and summarize the results
  - Topic: Application work of
    - ◆ E.g.:
      - Basic image processing
      - Image classification, Object detection
      - Related to your research topic, etc...
  - Notes
    - ◆ Please summarize the contents in a way that is easy for others to read and understand
    - ◆ Please summarize logically what problem you are dealing with, how you are going to do it, and what results you have obtained
    - ◆ No need source code
    - ◆ Not your research presentation
  - Pages: A4, 2~4 pages

# ○ Programming Environment

14

□ Language: Python

□ Environment: Google Colaboratory

- Cloud-based Jupyter notebook

## About Google Colaboratory

- The cloud-based Jupyter Notebook environment provided by Google
- Colab provides the use of free GPU/TPU
- Details: <https://colab.research.google.com/notebooks/intro.ipynb>

## ▾ About Python

- Interpreted programming language
- Open source libraries are provided for computer vision, machine learning, ...
- Simple structure and easy to use
- Don't care about data type for definition

```
[ ] 1 # Press [Shift+Enter] for operate Python code on google colab
    2 x = 10 # Define variable
    3 print("Hello World!!") # Define a string variable using single or double quotation
    4 print(123)
    5 print(0.123)
    6 print(x)
```

```
Hello World!!
123
0.123
10
```

## ▾ Color Histogram

- Brightness
- Contrast

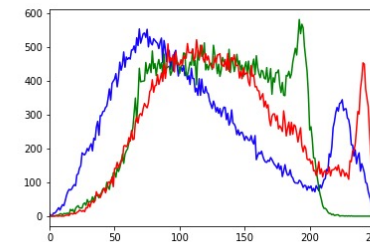
```
[ ] 1 imagePath = os.path.join(imageDir, "Mandrill.bmp")
```

```
[ ] 1 img = cv2.imread(imagePath, cv2.IMREAD_UNCHANGED)
    2 cv2_imshow(img)
```



```
[ ] 1 def colorHist(img):
    2     color = ('b','g','r')
    3     for i,col in enumerate(color):
    4         histr = cv2.calcHist([img],[i],[None],[256],[0,256])
    5         plt.plot(histr,color = col)
    6         plt.xlim([0,256])
    7     plt.show()
```

```
[ ] 1 colorHist(img)
```



# Basic Image Processing

---

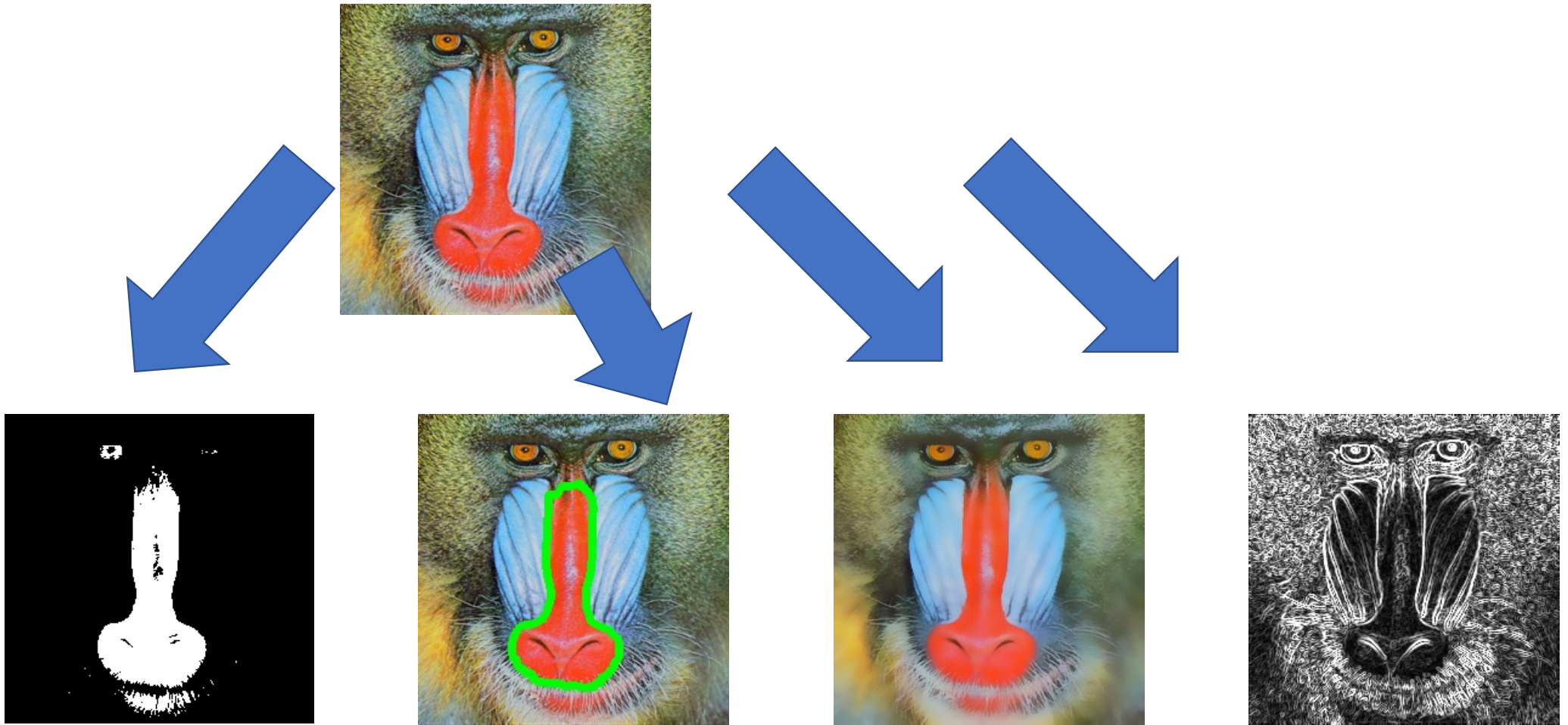
15

## □ Topic

- Color Histogram
- Binarization
- Smoothing
- Remove Salt-and-Pepper-Noise
- Edge Extraction

# ○ Basic Image Processing

16





# ○ Basic Image Processing

---

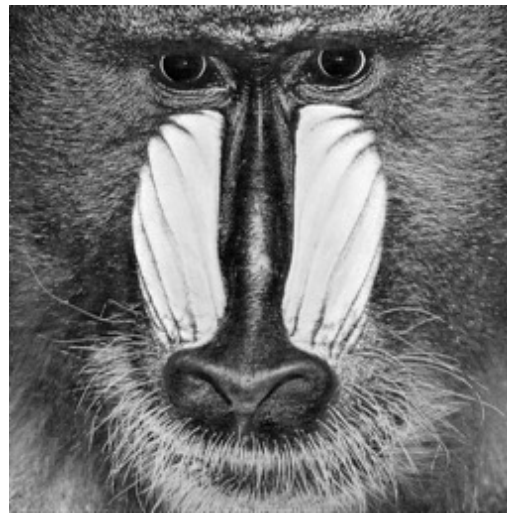
17

## □ Color Image

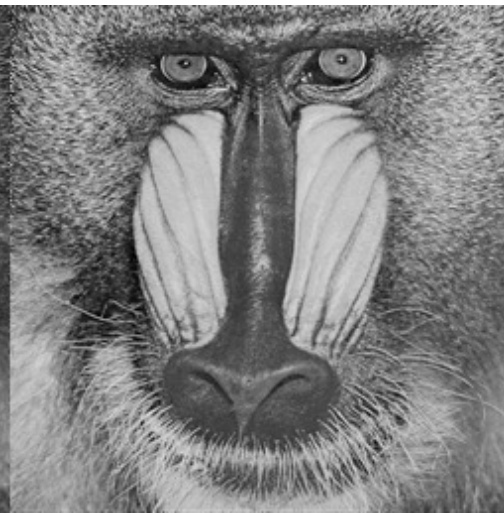
- Consists of three channel images (blue, green, red)



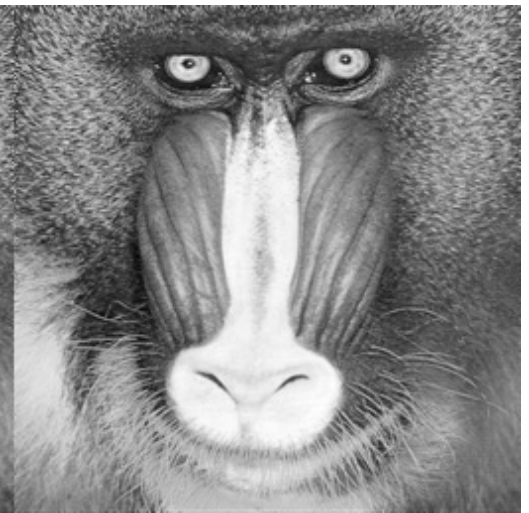
Color Image



Blue-Channel



Green-Channel



Red-Channel

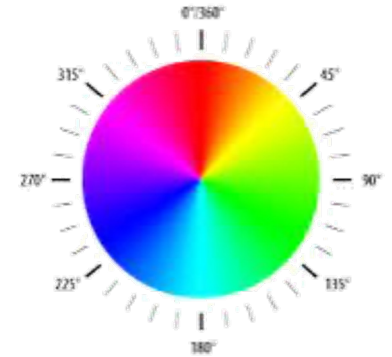
# Color Space

18

□ RGB (Red, Green, Blue)

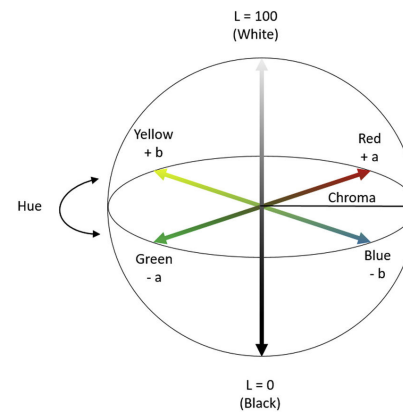
□ HSV

- Hue (色相): color [deg]
- Saturation (彩度):  $\max(R, G, B) - \min(R, G, B)$  or  $(\max - \min) / \max$
- Value (Brightness) (明度):  $\max(R, G, B)$



□ LAB

- L: (0) black -> (100) white
- a: (minus) green -> (plus) red
- b: (minus) blue -> (plus) yellow



# Color Histogram

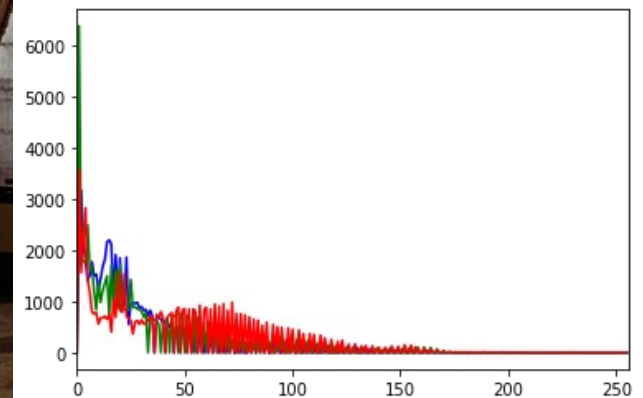
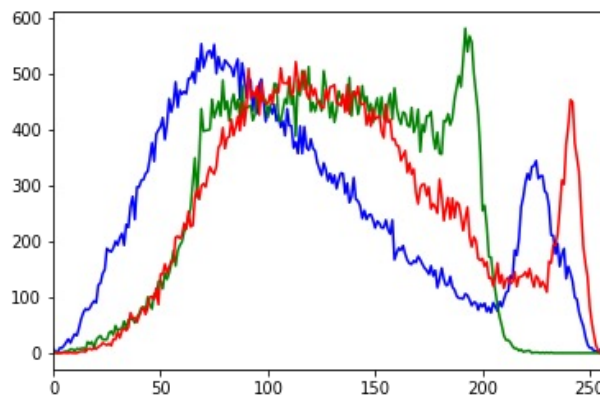
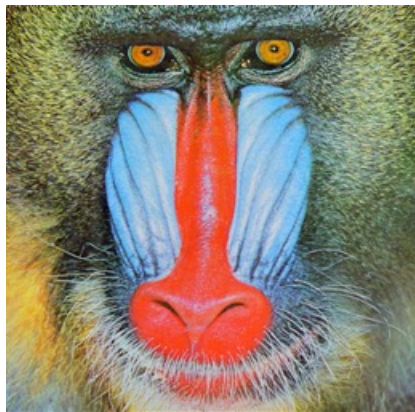
19

## □ Brightness

- Pixel value: lightness or darkness

## □ Contrast

- Difference in brightness in the image

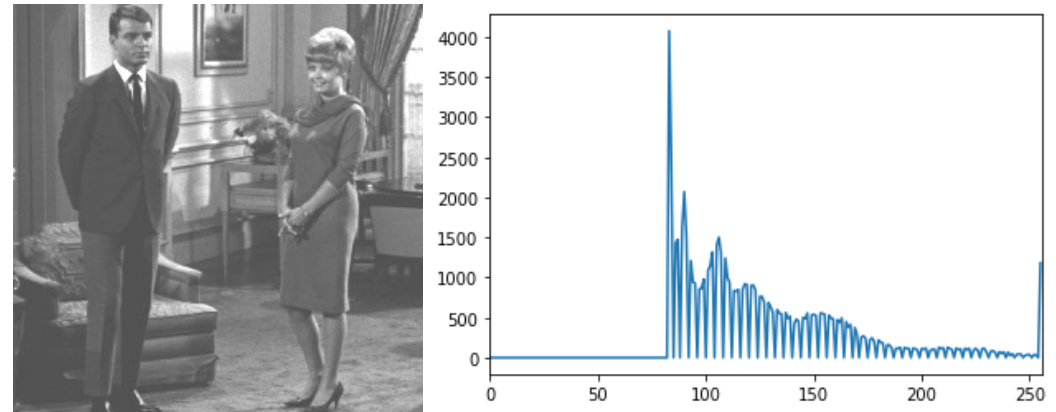
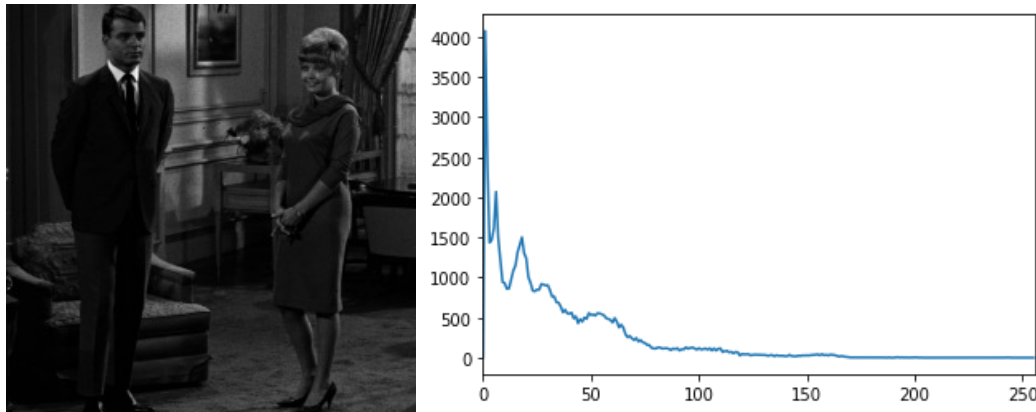


# ○ Normalize Brightness

20

□ Update pixel values based on the average pixel value and the standard deviation

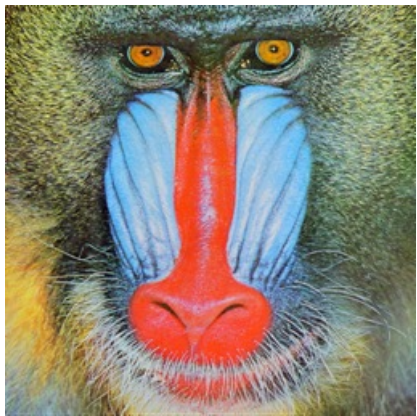
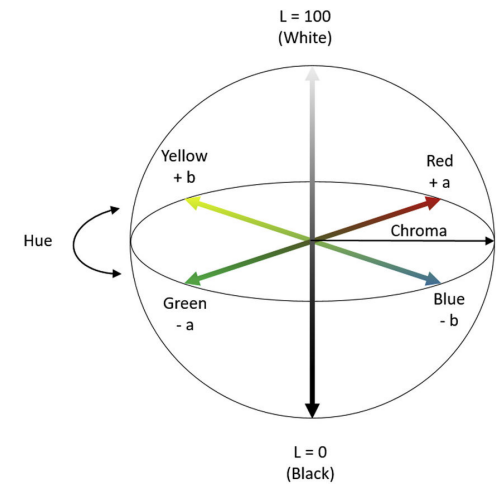
- E.g.  $x' = 127 \left( 1.0 + \frac{(x - \bar{x})}{n\sigma_x} \right)$ 
  - ◆  $x, x'$ : pixel value ( $0 \leq x, x' \leq 255$ )
  - ◆  $\bar{x}$ : average value
  - ◆  $\sigma_x$ : standard deviation
  - ◆  $n$ : gain value



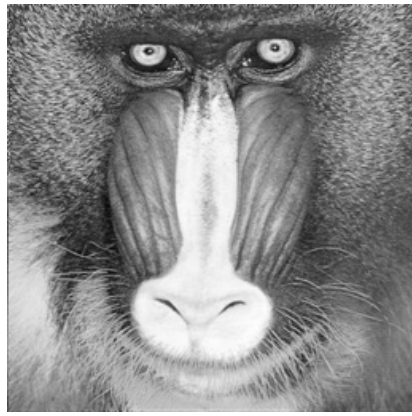
# Contour Extraction (1/2)

21

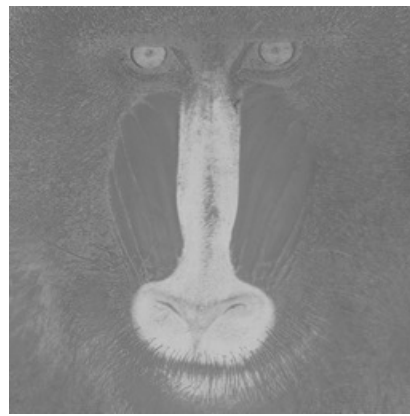
- Extract region for the binarized image
- E.g. extract the red nose
  - Prepare binarized image for A-channel image from Lab color space using Otsu's method



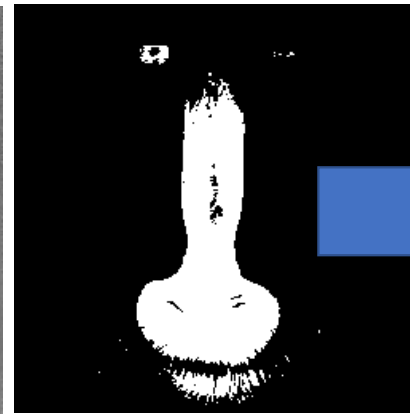
Original



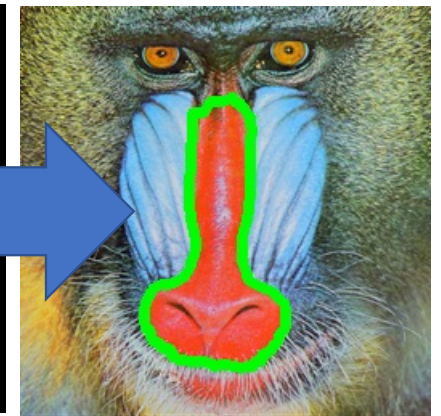
red-channel



A-channel



Binarized image



Output

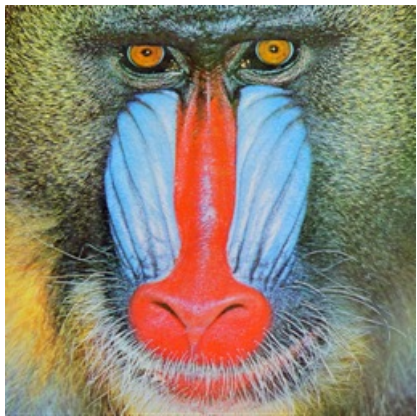


# ○ Contour Extraction (2/2)

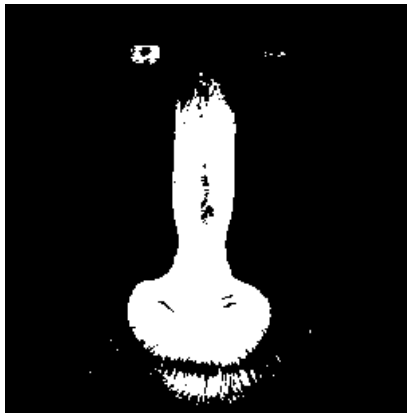
22

□ E.g. extract the red nose

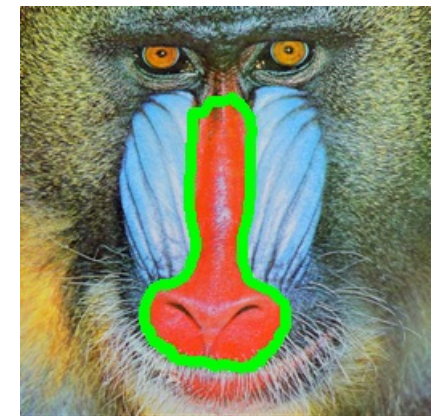
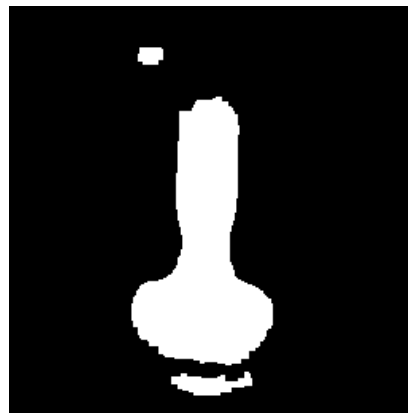
- Prepare binarized image for A-channel image from Lab color space using Otsu's method
- Apply morphology operation to fill the hole
  - ◆ Dilate operation: 3 times
  - ◆ Erode operation: 3 times
- Extract contour region with maximum contour area



Original



Binarized image



Output

# Binarization

23

- ▣ Divide pixel values based on the threshold
  - Manual threshold setting is not easy



Original



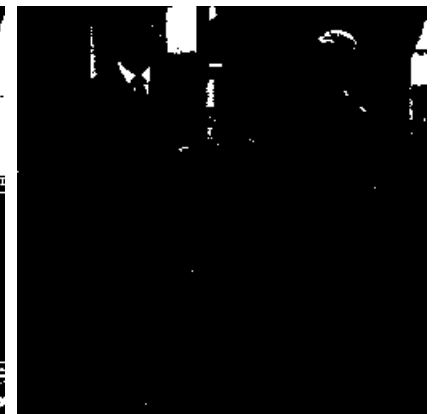
Threshold: 16



Threshold: 32



Threshold: 64



Threshold: 128



# ○ Binarization: Otsu's Method (1/2)

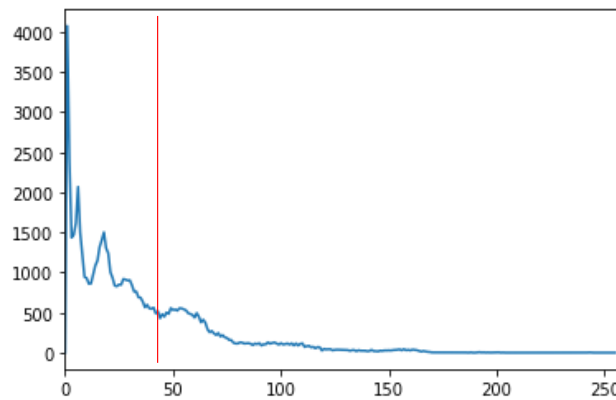
24

## □ Otsu's Method

- Threshold value is determined based on the pixel distribution
  - ◆ Find maximum inter-class variance



Original



Otsu's Method

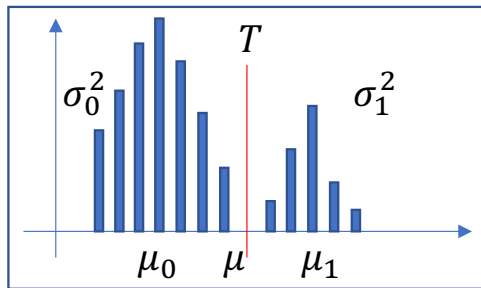
# ○ Binarization: Otsu's Method (2/2)

25

## □ Otsu's Method

- Threshold value is determined based on the pixel distribution

◆ Find maximum inter-class variance



Normalized histogram:  $p(i)$  ( $0 \leq i \leq 255$ )

$$\sum_{i \in [0, 255]} p(i) = 1$$

Class probability:  $\omega_0 + \omega_1 = 1$

$$\omega_0 = \sum_{i \in [0, T-1]} p(i) \quad \omega_1 = \sum_{i \in [T, 255]} p(i)$$

Mean:  $\mu = \omega_0 \mu_0 + \omega_1 \mu_1$

Intra-class variance:  $\sigma_w^2 = \omega_0 \sigma_0^2 + \omega_1 \sigma_1^2$

Inter-class variance:  $\sigma_B^2 = \omega_0 (\mu_0 - \mu)^2 + \omega_1 (\mu_1 - \mu)^2$   
 $= \omega_0 \omega_1 (\mu_0 - \mu_1)^2$

Total variance:  $\sigma^2 = \sigma_w^2 + \sigma_B^2$

### Hypothesis

- Class 0 and Class 1 should be separated (maximize  $\sigma_B^2$ )
- Each class variance should be small (minimize  $\sigma_w^2$ )

Maximize  $\frac{\sigma_B^2}{\sigma_w^2} = \frac{\sigma_B^2}{\sigma^2 - \sigma_B^2}$   Maximize  $\sigma_B^2$

# Morphology Operation

26

□ Input: binarized image

- Erode: expand black pixel
  - ◆ If there is black px in the kernel, the value is black
- Dilate: expand white pixel
  - ◆ If there is white px in the kernel, the value is white
- This operation can remove tiny noise



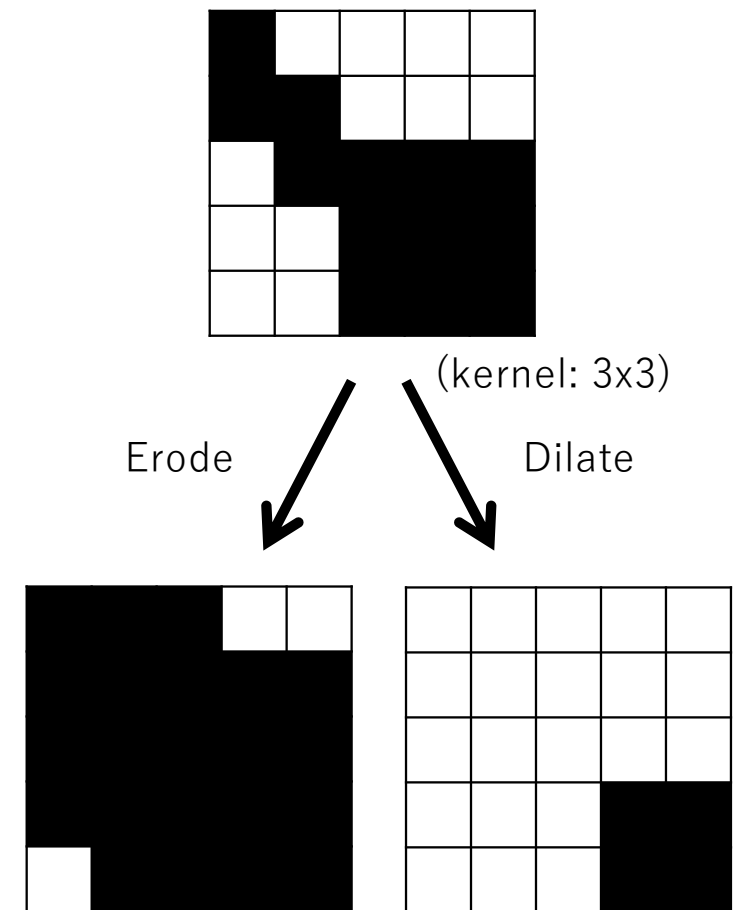
Original



Erode (kernel: 3x3)



Dilate (kernel: 3x3)



# Spatial filtering

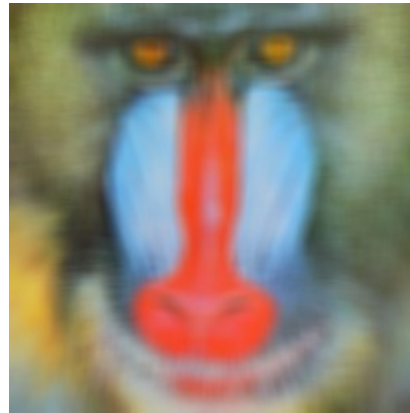
27



Original (256x256)

$$* \begin{pmatrix} & & \\ & & \\ & & \end{pmatrix} \rightarrow$$

$(2N + 1) \times (2N + 1)$



## □ Spatial filtering

- $\tilde{I}(x, y) = \sum_{j=-N}^N \sum_{i=-N}^N F(i, j) I(x + i, y + j)$

## □ Convolution

- $\tilde{I}(x, y) = \sum_{j=-N}^N \sum_{i=-N}^N F(i, j) I(x - i, y - j)$

- $\tilde{I} = F * I$

# Smoothing: Blur

28



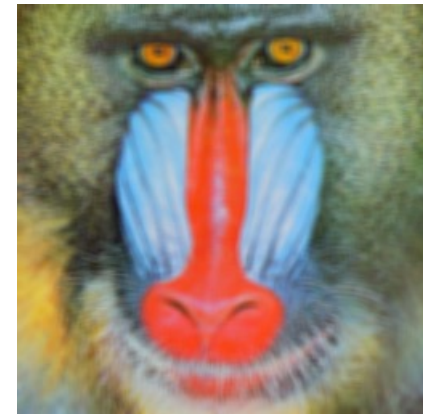
Original (256x256)

$$* \begin{pmatrix} & & \\ & & \\ & & \end{pmatrix} \rightarrow$$

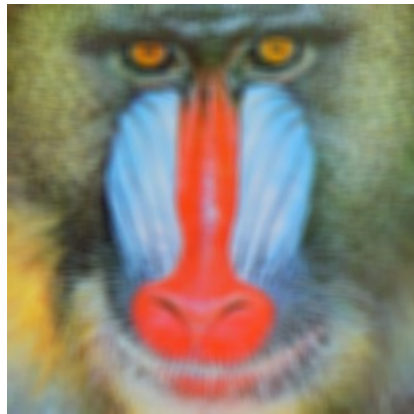
$(2N + 1) \times (2N + 1)$



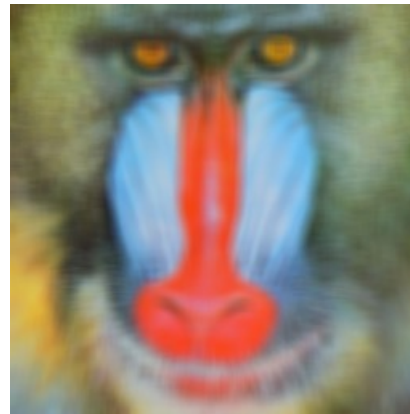
Box filter (kernel: 3x3)



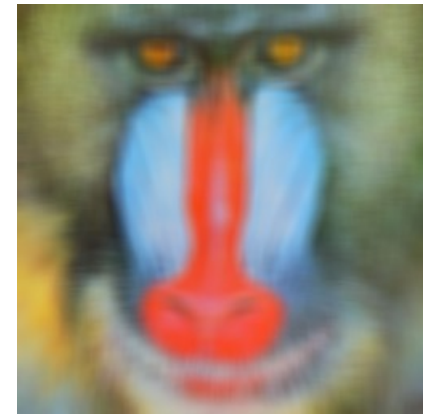
Box filter (kernel: 5x5)



Box filter (kernel: 7x7)



Box filter (kernel: 9x9)



Box filter (kernel: 11x11)



# ○ Salt-and-pepper noise

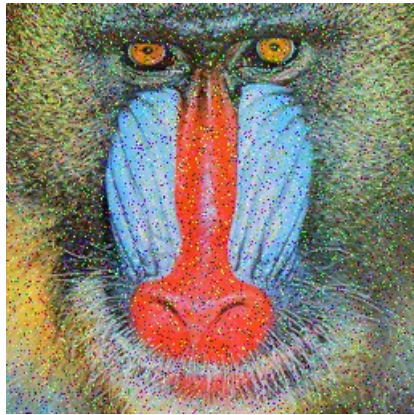
29

## □ Pixel level noise

- How to remove such noise?



Original (256x256)



Noised image



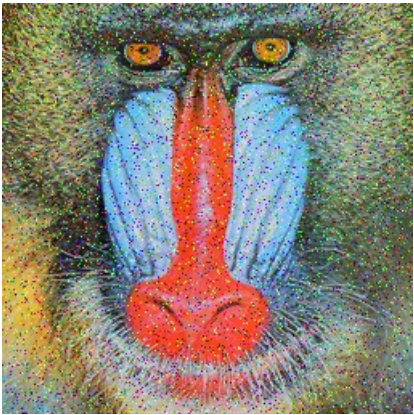


# Denoise: Median Filter

30

▣ Remove salt-and-pepper noise: Median Filter

- Find mean pixel for surrounding pixels



Noised image



Box filter (kernel: 5x5)



Median filter (kernel: 5x5)

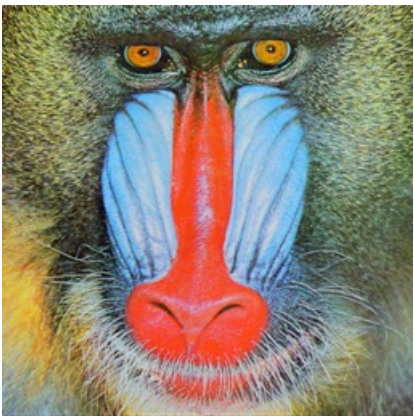
# Smoothing: Box Filter

31

## Image Blur: Box filter

- Compute average pixel value for surrounding pixels

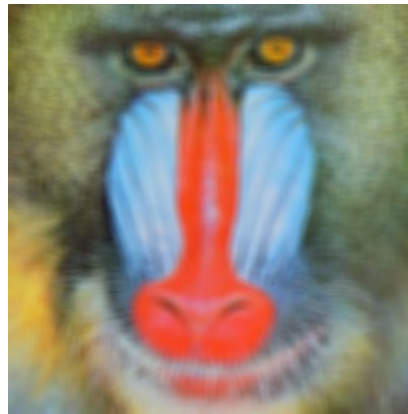
- Kernel(3x3):  $F = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$



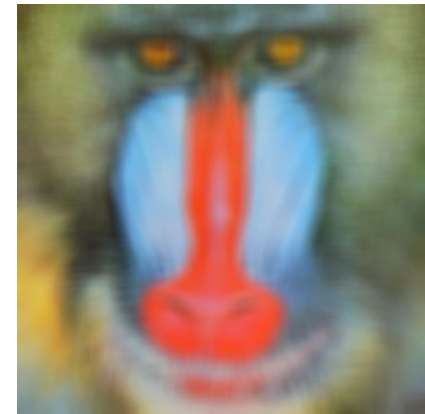
Original (256x256)



Box filter (kernel: 3x3)



Box filter (kernel: 7x7)



Box filter (kernel: 11x11)

# Smoothing: Gaussian Filter

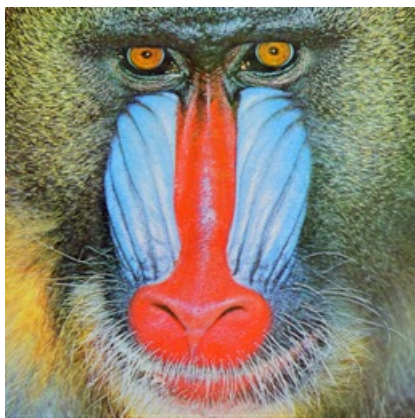
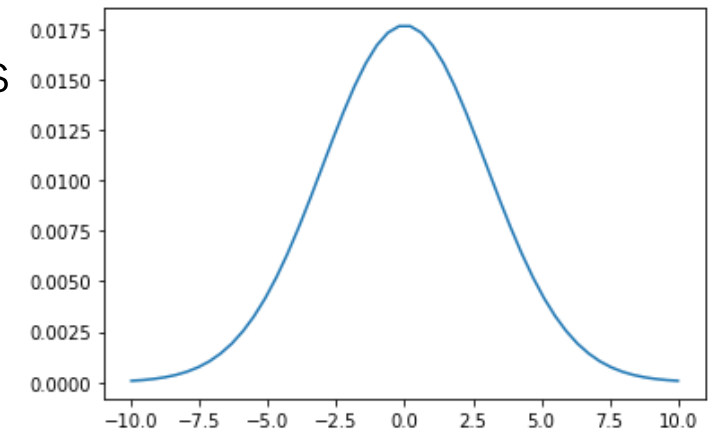
32

## □ Box filter

- Smoothing based on averaging surrounding pixels

## □ Gaussian filter

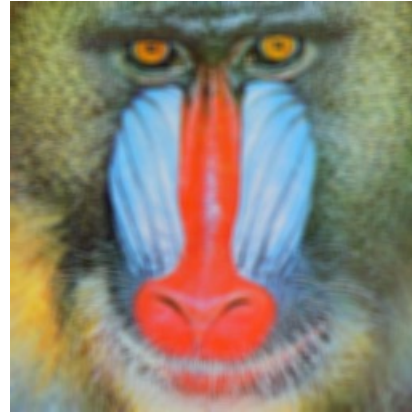
- Smoothing based on Gaussian distribution
- Kernel:  $G(i, j, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2+j^2}{2\sigma^2}\right)$
- $\tilde{I}(x, y) = \sum_i \sum_j G(i, j, \sigma) I(x + i, y + j) / \sum_i \sum_j G(i, j, \sigma)$



Original (256x256)



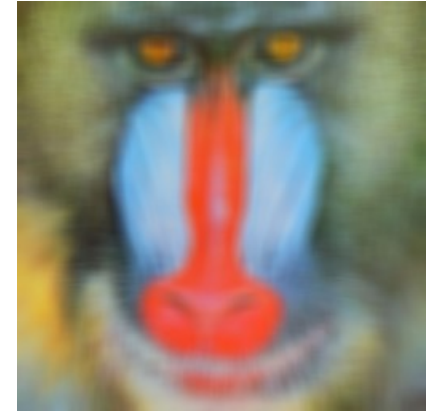
kernel: 5x5,  $\sigma=2$



kernel: 5x5,  $\sigma=10$



kernel: 11x11,  $\sigma=2$



kernel: 11x11,  $\sigma=10$

# Smoothing: Bilateral Filter (1/2)

33

## □ Gaussian filter

- kernel value is calculated based on spatial distance
- Gaussian function:  $G(i, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2}{2\sigma^2}\right)$
- Kernel:  $G(i, j, \sigma) = G(i, \sigma)G(j, \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2+j^2}{2\sigma^2}\right)$ 
  - ◆  $-2N \leq i, j \leq 2N$

## □ Bilateral filter

- kernel value is calculated based on spatial distance and color difference
- Kernel: 
$$W(x, y, i, j) = G(i, j, \sigma_d)G(I(x, y) - I(x + i, y + j), \sigma_c)$$
$$= \frac{1}{2\pi\sigma_d^2} \exp\left(-\frac{i^2 + j^2}{2\sigma_d^2}\right) \frac{1}{2\pi\sigma_c^2} \exp\left(-\frac{(I(x, y) - I(x + i, y + j))^2}{2\sigma_c^2}\right)$$

spatial distance                      Color difference
- $\tilde{I}(x, y) = \sum_i \sum_j W(x, y, i, j) I(x + i, y + j) / \sum_i \sum_j W(x, y, i, j)$

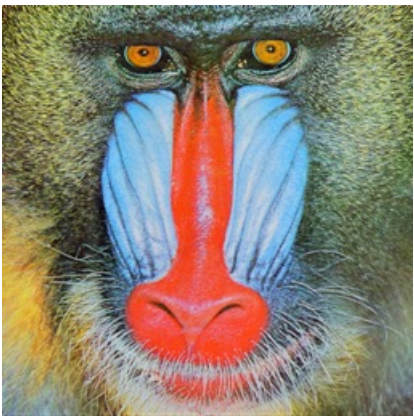


# Smoothing: Bilateral Filter (2/2)

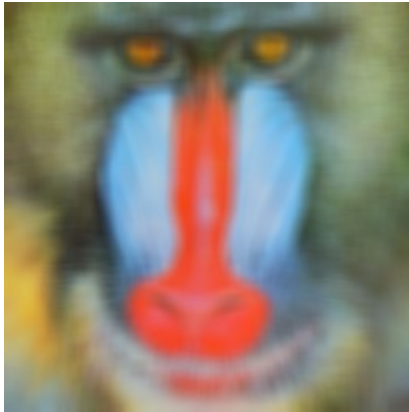
34

## □ Bilateral filter

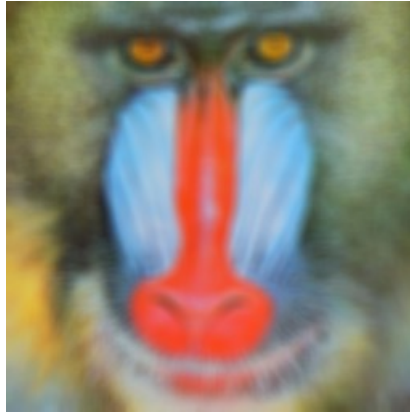
- kernel value is calculated based on spatial distance and color difference
- Kernel:  $W(x, y, i, j) = G(i, j, \sigma_d)G(I(x, y) - I(x + i, y + j), \sigma_c)$
- $\tilde{I}(x, y) = \sum_i \sum_j W(x, y, i, j)I(x + i, y + j) / \sum_i \sum_j W(x, y, i, j)$
- Smoothing while keeping edge information



Original (256x256)



Gaussian: 11x11,  $\sigma=10$



$\sigma_d=10, \sigma_c=100000$



$\sigma_d=10, \sigma_c=1$



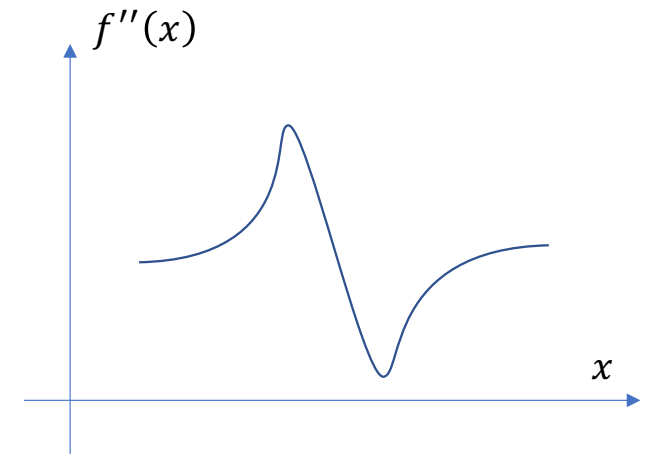
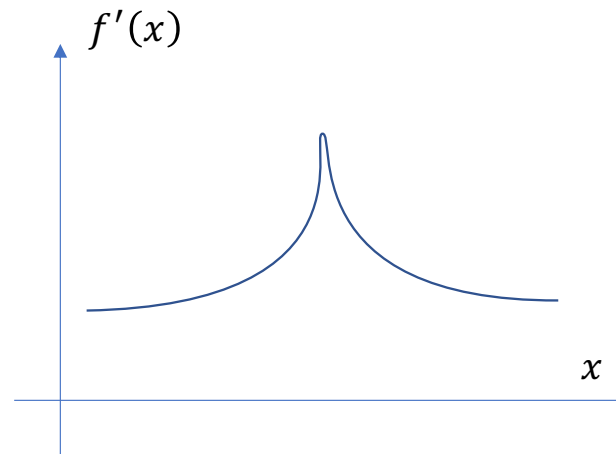
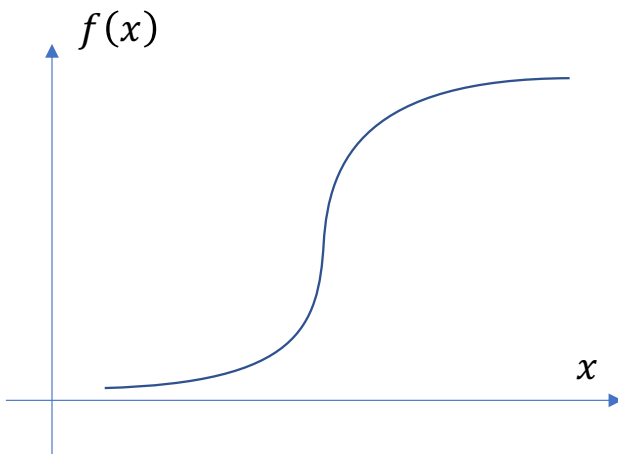
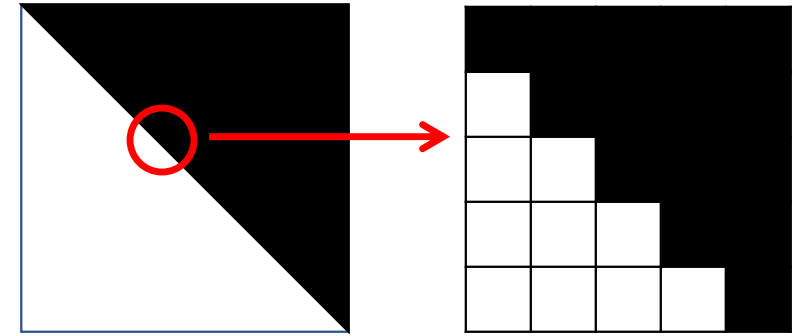
$\sigma_d=10, \sigma_c=100$

# Edge Extraction

35

## □ Edge extraction

- Emphasize pixel boundary





# Edge Extraction: 1st derivative

36

## □ Sobel filter

- 1st derivative for surrounding pixels

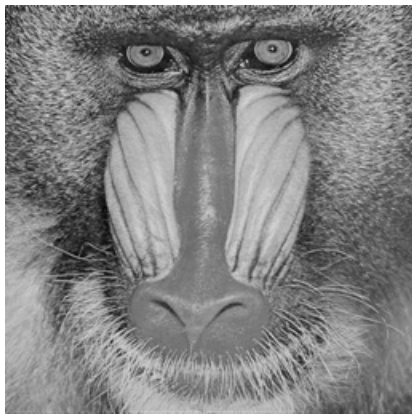
$$\blacklozenge f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

$\Delta x = 1 \text{ or } -1$

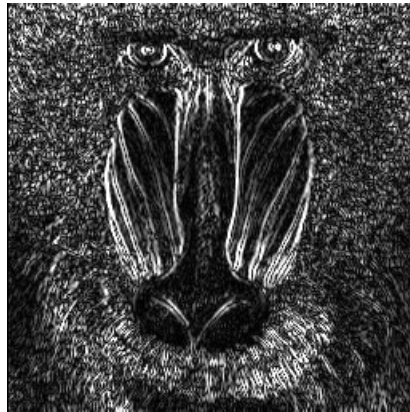
$$f'(x) = f(x+1) - f(x) \text{ or } f(x) - f(x-1)$$

$$\blacklozenge f'(x) = (f(x+1) - f(x-1))/2$$

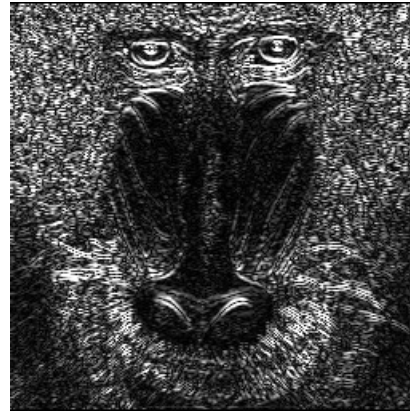
- Kernel(3x3):  $F_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, F_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$



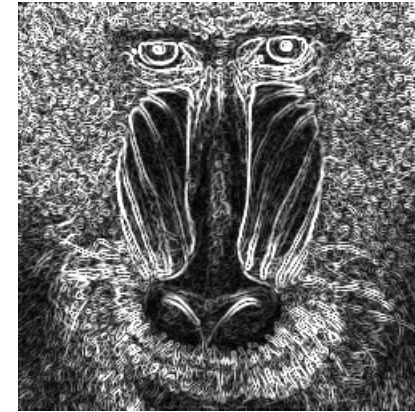
Original (256x256)



Sobel (x, abs)



Sobel (y, abs)



Sobel ( $\sqrt{x^2 + y^2}$ )

# Edge Extraction: 2nd derivative (1/2)

37

## □ Laplacian filter

- 2nd derivative for surrounding pixels

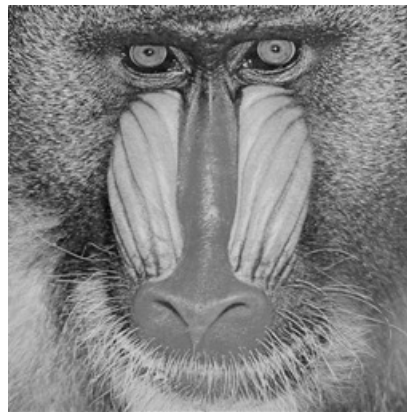
◆  $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} = I_{xx}(x, y) + I_{yy}(x, y)$

◆  $I_{xx}(x, y) = \{I(x+1, y) - I(x, y)\} - \{I(x, y) - I(x-1, y)\} = I(x+1, y) + I(x-1, y) - 2I(x, y)$

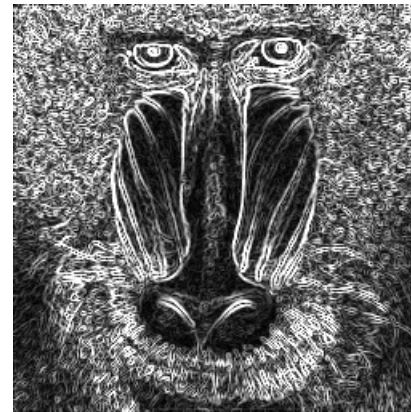
◆  $I_{yy}(x, y) = I(x, y+1) + I(x, y-1) - 2I(x, y)$

◆ Kernel:  $F = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$

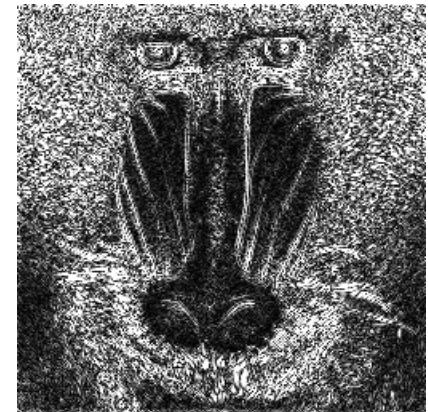
◆ Kernel:  $F = \begin{pmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{pmatrix}$



Original (256x256)



Sobel ( $\sqrt{x^2 + y^2}$ )



Laplacian (abs)

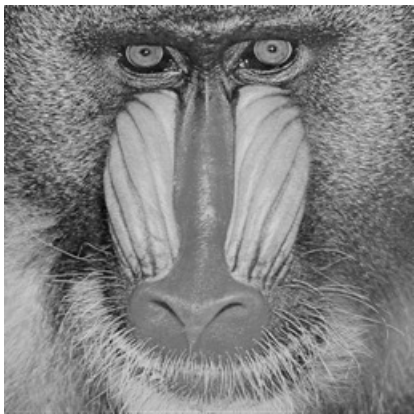


# Edge Extraction: 2nd derivative (2/2)

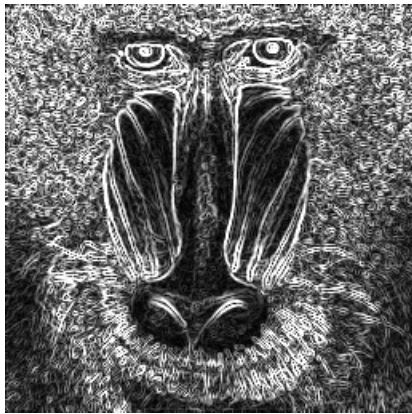
38

## □ Laplacian of Gaussian

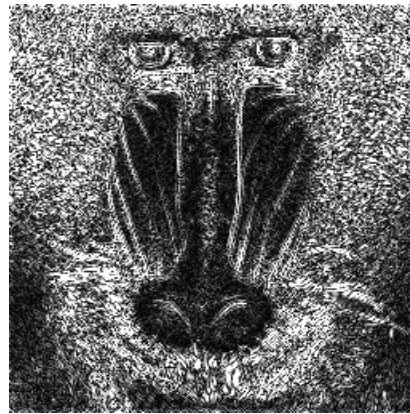
- Apply gaussian filter before derivative
  - ◆ Suppress tiny noise of derivative
- $\nabla^2(G(x, y, \sigma) * I(x, y)) = (\nabla^2 G(x, y, \sigma)) * I(x, y)$
- $\nabla^2 G = \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2} = -\frac{1}{\pi\sigma^4} \left(1 - \frac{x^2+y^2}{2\sigma^2}\right) \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$



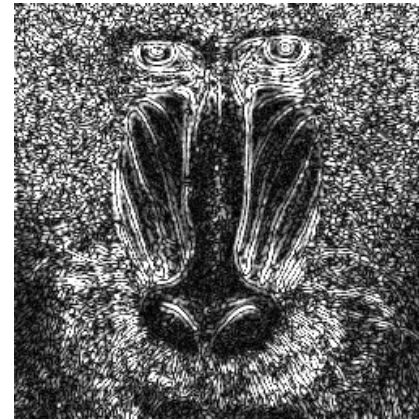
Original (256x256)



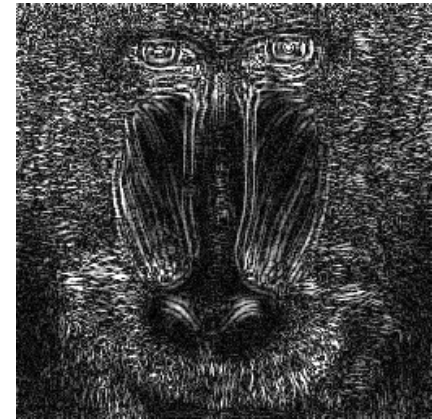
Sobel (abs)



Laplacian (abs)



LoG ( $\sigma=1$ )



LoG ( $\sigma=5$ )